

# README\_dev.md – tillägg / ändringar

## Now / How to test

- Kör alla tester lokalt

```
./gradlew test
```

- Kör snabb “smoke”

```
./gradlew test --tests org.dcsim.unit.MiniminiSolverTests --tests  
org.dcsim.unit.StampMiniTests --tests org.dcsim.unit.SolverMiniTests  
--tests org.dcsim.unit.IslandSeparationMiniTests
```

## Verbosity (opt-in)

- Default: tyst körning.
- Aktivera extra loggar när du felsöker:

```
./gradlew test -Ddcsim.verbose=true
```

## Graph export (opt-in)

- Skriv ut DOT-filer endast när begärt via flagga (exempel):

```
./gradlew test -Ddcsim.graph=topology
```

- Rendera till PNG (kräver Graphviz dot ):

```
dot -Tpng build/graphs/<name>.dot -o build/graphs/<name>.png
```

## Java & OS

- Stödda CI-miljöer: Java 17 (och ev. 21) på Linux/Windows.

## README\_dev — Arbetsflöde för patchar, Git & IntelliJ (dc-simulator)

Den här guiden är skriven för **just det här projektet**. Målet är att göra vardagen lugnare och mer förutsägbar när vi testar patchar, felsöker och backar om något går snett.

Kort filosofi: **små steg, egna brancher, tydliga diffar, lätt att backa.**

---

# 1) Snabbstart — det här gör du *varje gång* innan du testar något nytt

```
# 1) Se till att du är uppdaterad
git fetch origin
git checkout main
git pull

# 2) Skapa en ny trygg branch
git checkout -b feature/<kort-namn>

# 3) (valfritt) Spara en baseline-diff
git diff > baseline.diff
```

**Varför?** Då kan du när som helst:

- `git reset --hard` till senaste commit,
- kasta lokalen och börja om från en ren branch,
- eller `git apply` en patch igen om det behövs.

---

## 2) Ta emot patchar (från mig) på ett tryggt sätt

Du får en fil `0001-namn.patch` (eller flera). Applicera så här:

```
git checkout -b feature/apply-patch-XYZ
git apply --3way 0001-namn.patch # --3way hjälper vid mindre diffar från din
bas
# Om inga fel: bygg
./gradlew build
```

Om `git apply` klagar:

- Kör `git status` och se vilka filer som är “unmerged”.
- Öppna dem i IntelliJ och lös konflikter visuellt (du får pilar <<<<< >>>>>).
- Spara, `git add` filerna, sedan `git commit -m "Resolve patch conflicts"`.

**Tips:** Vill du provläsa patchen först?

```
git apply --stat 0001-namn.patch # sammanfattning
git apply --check 0001-namn.patch # testkörning utan att ändra filer
```

---

## 3) Backa när det brinner

### A. Backa lokala ändringar i en FIL

```
git restore -- src/main/java/org/dcsim/actors/GridModelActor.java
```

### B. Backa allt i repos lokalt (till senaste commit)

```
git reset --hard HEAD
```

### C. Gå tillbaka till en tidigare commit

```
git log --oneline --graph --decorate
git checkout <commit-id>    # går till det läget (detta är "detached HEAD")
# eller skapa en branch där:
git checkout -b debug/<namn> <commit-id>
```

### D. Jag vill bara kasta min experiment-branch

```
git checkout main
git branch -D feature/experiment-X
```

---

## 4) IntelliJ — de tre viktigaste sakerna

### 4.1 Local History (livräddare)

- Högerklicka på en fil eller katalog → **Local History** → **Show History**.
- Du kan jämföra och rulla tillbaka till valfri punkt (även om du inte committat).

### 4.2 Jämför två versioner

- Project-vyn: markera filen → **Right-click** → **Compare With**
- Eller välj två filer/brancher och använd **Compare With Each Other**.

### 4.3 Applicera patch i IntelliJ (grafiskt)

- VCS menyn → **Apply Patch**  
→ välj `.patch` → följ guiden.
-

## 5) Körning, logg & debug-flagga

### 5.1 Starta appen

- **Run configuration:** `DcSimApp.main()`
- **Program-argument:** `project/3subs1train/application.conf`
- **VM-options (verbose):**

```
-Ddcsim.verbose.all=true
```

### 5.2 Var loggen hamnar

- Console i IntelliJ + filen `output/electrical_application.csv`
- Vid behov, loggzip som `logg.zip` (export ur appen).

## 6) Var sätter jag breakpoints & debug-prints?

Målet: följ *energiflödet* för ett tick.

### 6.1 TrainActor — första sanningen (profildata → kraft)

Efter att du räknat fram mot/brake/aux för tick:et, lägg print:

```
System.out.printf("[TA] id=%s mot=%.3f kW brk=%.3f kW aux=%.3f kW\n",  
    trainId, motoringKW, brakingKW, auxiliaryKW);
```

### 6.2 GridModelActor — innan första solve

Direkt efter att du kört:

```
tl.setRequestedComponents(u.motoringKW, u.brakingKW, u.auxiliaryKW);
```

bygg totaler i watt och skicka:

```
java.util.Map<String, Double> reqW_direct = new java.util.LinkedHashMap<>();  
for (var e : latest.entrySet()) {  
    var id = e.getKey();  
    var u = e.getValue();  
    double pTotW = (u.motoringKW + u.brakingKW + u.auxiliaryKW) * 1000.0;  
    reqW_direct.put(id, pTotW);  
    System.out.printf("[GMA-A] id=%s mot=%.3f kW brk=%.3f kW aux=%.3f kW -> push  
%.1f W\n",  
        id, u.motoringKW, u.brakingKW, u.auxiliaryKW, pTotW);
```

```
}
this.solver.setTrainRequestedPower(reqW_direct, this.trainDtSec);
```

## 6.3 (om du har steg 2 med regen-klippning)

Efter att du räknat fram `factor` och `allowedNetW`:

```
java.util.Map<String, Double> reqW_direct2 = new java.util.LinkedHashMap<>();
for (var e : latest.entrySet()) {
    var id2 = e.getKey();
    var u2 = e.getValue();
    double allowedNetW2 = netW0.getOrDefault(id2, 0.0) * factor;
    double regenKW2 = -allowedNetW2 / 1000.0;
    double pTotW2 = (u2.motoringKW + regenKW2 + u2.auxiliaryKW) * 1000.0;
    reqW_direct2.put(id2, pTotW2);
    System.out.printf("[GMA-B] id=%s mot=%.3f kW brkAdj=%.3f kW aux=%.3f kW ->
push %.1f W (factor=%.6f)%n",
        id2, u2.motoringKW, regenKW2, u2.auxiliaryKW, pTotW2, factor);
}
this.solver.setTrainRequestedPower(reqW_direct2, this.trainDtSec);
```

## 6.4 DclterativeAdapterSolver — tog solvern emot och applicerade?

I `setTrainRequestedPower(...)` lägg:

```
System.out.printf("[ADAPT] direct requestedPowerW: %s%n", requestedPowerW);
```

Och i appliceringen, precis innan `t1.setRequestedPower(...)`:

```
System.out.printf("[ADAPT] apply TL id=%s p=%.1f W (map=%s)%n",
    t1.getId(), p != null ? p : Double.NaN, lastRequestedPowerW);
```

## 6.5 TrainLoad — blev värdet kvar?

Lägg i `setRequestedPower(Real rp)`:

```
System.out.printf("[TL] %s setRequestedPower=%.1f W%n", this.id, rp != null ?
rp.asDouble() : -1.0);
```

och i `computeCurrent(...)` (eller motsv):

```
System.out.printf("[TL] %s compute: Va=%.1f V Vb=%.1f V req=%.1f W cutoff=%.1f
V%n",
```

```
this.id, Va, Vb, requestedPower != null ? requestedPower.asDouble() : -1.0,
this.cutoffVoltage);
```

## 7) Checklistor när något blir 0

- **[TA]** visar 0 → profilen/beräkningen ger 0. Kontrollera filväg, mallparametrar, tidsindexering.
- **[GMA-A]** visar 0 → värdet nollas i aktorn innan solvern matas (fel ordning eller latest saknas).
- **[ADAPT]** visar icke-noll men `[TL] setRequestedPower` visar 0 → id-mismatch (ex. `Train1` vs `T1`). Justera nycklarna (håll dem identiska).
- **[TL compute]** visar att cutoff/minVoltage slår till → höj cutoff, eller se till att station/spänningsnivå ger > cutoff.

Snabbt röktest: tryck in 200 kW temporärt innan push:

```
if (reqW_direct.containsKey("T1") && Math.abs(reqW_direct.get("T1")) < 1e-6)
{
    reqW_direct.put("T1", 200_000.0);
    System.out.println("[GMA-A TEST] forcing T1=200kW this tick");
}
```

## 8) Vanliga Git-kommandon (fuskklapp)

```
# Visa status
git status

# Visa ändringar i arbetskatalogen
git diff

# Stage & commit
git add -A
git commit -m "Beskrivning"

# Pusha din branch
git push -u origin feature/<namn>

# Byt branch
git checkout main
git checkout feature/<namn>
```

```
# Radera lokal branch (som du inte behöver längre)
```

```
git branch -D feature/<namn>
```

---

## 9) Förslag på rutin varje dag

1. `git fetch` & `git pull` på `main`.
2. Skapa ny branch för dagens jobb.
3. Applicera 1 patch / 1 ändring i taget → bygg → kör → logga.
4. Om något blir kaos → `git reset --hard` och börja om från ren branch.
5. När det funkar → `git commit`, `git push`, PR.